

Práctica 1.3. Procesos

Objetivos

En esta práctica se revisan las funciones del sistema básicas para la gestión de procesos: políticas de planificación, creación de procesos, grupos de procesos, sesiones y ejecución de programas.

Contenidos

Preparación del entorno para la práctica
Políticas de planificación
Procesos, grupos de procesos y sesiones
Ejecución de programas

Preparación del entorno para la práctica

Algunos de los ejercicios de esta práctica requieren permisos de superusuario para poder modificar algunos atributos de un proceso, ej. políticas de tiempo real. Por este motivo, es recomendable realizarla en una **máquina virtual** en lugar de las máquinas físicas del laboratorio.

Para lanzar la máquina virtual en los laboratorios:

1. Si existen máquinas virtuales creadas con anterioridad, borrar las máquinas virtuales existentes ejecutando el siguiente comando en la consola:

```
$ rm -rf $HOME/VirtualBox\ VMs/
```

2. En los equipos del laboratorio, ir al directorio /mnt/DiscoVMs/ASOR usando el gestor de ficheros (hay un acceso directo en el escritorio) y localizar el fichero ASOR-FE.ova (en otros equipos, descargar el fichero [ASOR-FE.ova](#)). Abrir el fichero haciendo doble-clic y pulsar "Terminar" para importar la máquina virtual base ASOR-FE en VirtualBox. Alternativamente, se puede usar la opción "Importar servicio virtualizado" desde VirtualBox.
3. Lanzar la máquina virtual importada a través de VirtualBox. Para esta práctica en concreto, comprobar los recursos asignados (RAM, CPUs, etc.) y configurarlos adecuadamente antes de inicializarla.

Nota: Si estás usando WSL o docker, es posible que algunos de los ejercicios no funcionen correctamente.

Políticas de planificación

En esta sección estudiaremos los parámetros del planificador de Linux que permiten variar y consultar la prioridad de un proceso. Veremos tanto la interfaz del sistema como algunos comandos importantes.

Ejercicio 1. La política de planificación y la prioridad de un proceso puede consultarse y modificarse con `chrt(1)`. Adicionalmente, `nice(1)` y `renice(1)` permiten ajustar el valor de *nice* de un proceso. Consultar la página de manual de ambos comandos y comprobar su funcionamiento cambiando el valor de *nice* de la *shell* a -10 y después cambiando su política de planificación a `SCHED_FIFO` con

prioridad 12.

Ejercicio 2. Escribir un programa que muestre la política de planificación (como cadena) y la prioridad del proceso actual, además de mostrar los valores máximo y mínimo de la prioridad para la política de planificación. Prueba a utilizar tanto `sched_getparam(2)` como `getpriority(2)` para obtener valores de prioridad. ¿Se obtiene el mismo resultado?

Ejercicio 3. Ejecutar el programa anterior en una *shell* con prioridad 12 y política de planificación SCHED_FIFO como la del ejercicio 1. ¿Cuál es la prioridad en este caso del programa? ¿Se heredan los atributos de planificación?

Procesos, grupos de procesos y sesiones

Los grupos de procesos y las sesiones simplifican la gestión que realiza la *shell*, ya que permite enviar de forma efectiva señales a un grupo de procesos (suspender, reanudar, terminar...). En esta sección veremos esta relación y estudiaremos el interfaz del sistema para controlarla.

Ejercicio 4. `ps(1)` permite ver los procesos del sistema y su estado. Estudiar la página de manual y:

- Mostrar todos los procesos del usuario actual en formato extendido.
- Mostrar los procesos del sistema, incluyendo el identificador del proceso, el identificador del grupo de procesos, el identificador de sesión, el estado y el comando con todos sus argumentos.
- Observar el identificador de proceso, grupo de procesos y sesión de los procesos. ¿Qué identificadores comparten la *shell* y los programas que se ejecutan en ella? ¿Cuál es el identificador de grupo de procesos cuando se crea un nuevo proceso?

Ejercicio 5. Escribir un programa que muestre los identificadores del proceso (PID, PPID, PGID y SID), el número máximo de ficheros que puede abrir y su directorio de trabajo actual.

Ejercicio 6. Un demonio es un proceso que se ejecuta en segundo plano para proporcionar un servicio. Normalmente, un demonio está en su propia sesión y grupo. Para garantizar que es posible crear la sesión y el grupo, el demonio crea un nuevo proceso para crear la nueva sesión y ejecutar la lógica del servicio. Escribir una plantilla de demonio (creación del nuevo proceso y de la sesión) en el que únicamente se muestren los atributos del proceso (como en el ejercicio anterior). Además, establecer el directorio de trabajo del demonio a `/tmp`.

¿Qué sucede si el proceso padre termina antes que el hijo (observar el PPID del proceso hijo)? ¿Y si el proceso que termina antes es el hijo (observar el estado del proceso hijo con `ps(1)`)?

Introduce llamadas a `sleep(3)` en ambos procesos (padre e hijo) antes de que impriman sus respectivos mensajes. Observa con `ps/top/htop` los IDs de proceso, grupo y sesión, así como el terminal asociado. Antes de que terminen sus llamadas a `sleep()` presiona Ctrl-C en el terminal. ¿Qué ocurre? ¿Mueren ambos procesos?

Nota: Usar `sleep(3)` o `pause(3)` para forzar el orden de finalización deseado.

Ejecución de programas

Ejercicio 7. Escribir dos versiones, una con `system(3)` y otra con `execvp(3)`, de un programa que ejecute otro programa que se pasará como argumento por línea de comandos. En cada caso, se debe imprimir la cadena "El comando terminó de ejecutarse" después de la ejecución. ¿En qué

casos se imprime la cadena? ¿Por qué?

Nota: Considerar cómo deben pasarse los argumentos en cada caso para que sea sencilla la implementación. Por ejemplo: ¿qué diferencia hay entre `./ej7 ps -e1` y `./ej7 "ps -e1"`?

Ejercicio 8. Usando la versión con `execvp(3)` del ejercicio anterior y la plantilla de demonio del ejercicio 6, escribir un programa que ejecute cualquier programa como si fuera un demonio. Además, redirigir los flujos estándar asociados al terminal usando `dup2(2)`:

- La salida estándar al fichero `/tmp/daemon.out`.
- La salida de error estándar al fichero `/tmp/daemon.err`.
- La entrada estándar a `/dev/null`.

Comprobar que el proceso sigue en ejecución tras cerrar la *shell*.